CS3213: Foundations of Software Engineering

In-class Lecture and Exercises

Bioblitz

- 566 observations
- 197 species
- 6 observers (+1!)
- 119 identifiers

Black-bearded Gliding Lizard

Banded Pig

Peacock River Stingray

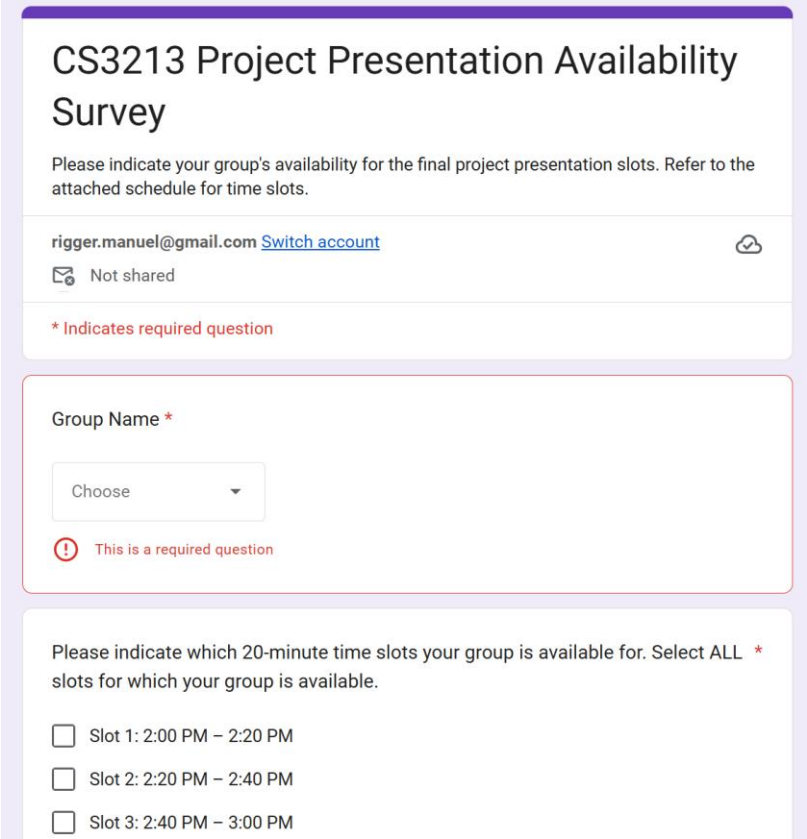
Lesser Mouse-deer

https://www.inaturalist.org/projects/nus-cs3213-2025

Mid-term Peer Evaluation

Course:	[CS3213-2026] Foundations of Software Engineering
Feedback Session Name:	Group Project Peer Evaluation for CS3213 (Mid-term)
Deadline:	Mon, 23 Mar 2026, 11:59 PM SGT
Session Instructions:	This peer evaluation feedback is part of the course assessment process and is designed to collect anonymous peer evaluations for group-based coursework. Your responses will be used to support fair and informed grading decisions for the intermediate and final reports. Participation in this peer evaluation is required as part of the course requirements. You will be asked to evaluate the **contribution**, **reliability**, and **communication** of your group members. You should provide honest, professional, and evidence-based ratings. Your responses will remain anonymous to other students. However, the course teaching staff (including the instructor and teaching assistants) will be able to access the submitted data for grading and moderation purposes. Individual responses may be reviewed in cases where clarification or grade adjustment is necessary. The collected data will be used solely for academic evaluation within this course and will not be shared outside the teaching team. By proceeding, you acknowledge that you understand the purpose of this evaluation, how your responses will be used, and agree to participate accordingly. Thank you for your collaboration!

Week 13: Project Presentations



CS3213 Project Presentation Availability Survey

Please indicate your group's availability for the final project presentation slots. Refer to the attached schedule for time slots.

rigger.manuel@gmail.com Switch account

Not shared

* Indicates required question

Group Name *

Choose

This is a required question

Please indicate which 20-minute time slots your group is available for. Select ALL slots for which your group is available. *

- [ ] Slot 1: 2:00 PM – 2:20 PM
- [ ] Slot 2: 2:20 PM – 2:40 PM
- [ ] Slot 3: 2:40 PM – 3:00 PM

Reminder: Attend Tutorials

Last Tutorial

W1	W2	W3	W4	W5	W6	Recess	W7	W8	W9	W10	W11	W12	W13
L1	L2	L3	L4	L5				L6	L7	L8	L9	L10	
	T1		T2				T3		T4			T5	

Last tutorial will be on April 10!

Mid-term Exam

National University of Singapore

CS3213—Foundations of Software Engineering

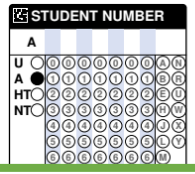
Mid-term Exam

Semester 2, 2025/2026

Time allowed: 1 hour

INSTRUCTIONS TO STUDENTS

1. Write down your **student number** on the right and using ink or pencil, shade the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME!
2. For each question that specifies the answers next to a cir-



Some minor point increases

printed on **FOUR (4) sheets of paper** including this cover page.

5. The time allowed for solving this test is **1 hour**.
6. Students are required to answer ALL questions.
7. The minimum amount of points for a question is 0 (i.e., there are no negative points).
8. This is a CLOSED BOOK assessment. Only a double-sided A4 helpsheet is allowed.

Qns	Marks
S1	/ 20
S2	/ 32
S3	/ 24
S4	/ 24
Total	/100

Agenda

- Q/A with Sau Sheong
- Recap
- Exercise slicing and delta debugging
- Delta debugging for isolating bug-inducing changes

Q/A with Sau Sheong

Next Week

- No (pre-)recorded lectures next week
- Instead: Q/A with Sau Sheong
- Also part of the exam!

GovTech (Chang Sau Sheong)

Deputy Chief Executive and Chief Technology Officer
GovTech Singapore · Full-time
Feb 2023 - Present · 3 yrs 2 mos
Singapore

GovTech is the government agency driving Singapore's Smart Nation initiative and public sector digital transformation. I lead GovTech teams to look at future technologies, set the technology standards and develop and deliver digital products for the Singapore government.

Operating Partner, Digital
Temasek · Full-time
Jul 2022 - Feb 2023 · 8 mos
Singapore

I work with portfolio companies management teams to deliver value. During my time in Temasek, I was assigned to the AI Strategy and Solutions team and helped a few ventures, including Minden.AI's Yuu loyalty coalition, AIcadium, an AI technology company dedicated to creating and scaling AI solutions for enterprises, and others. I was also involved in AI technology funds, direct investments, and technical due diligence on potential investments.

AI

GOVTECH SINGAPORE

From vibe coding to agentic engineering

How AI is reshaping software engineering in the Singapore government, and what comes next

Sau Sheong · Follow · 44 min read · Feb 28, 2026

I've been having a lot of conversations lately. With engineers, with leaders in the industry, with people building tools that didn't exist six months ago. And one thing has become very clear to me. AI isn't just changing software engineering. It's reshaping the whole thing, from how we write code to how we think about what code even is.

Generated by Nano Banana 2 from an original photo by Marvin Meyer on Unsplash

https://sausheong.com/from-vibe-coding-to-agentic-engineering-1ca3ca72b5ac

Active Programmer!

Chang Sau Sheong
sausheong

Follow

I write, code.

1.2k followers · 1 following

GovTech Singapore
Singapore
sausheong@gmail.com
https://sausheong.com
https://orcid.org/0009-0003-8918-0668
@sausheong

Popular repositories

gwp — Public
Go Web Programming code repository
JavaScript ☆ 1.6k 759

polyglot — Public
Polyglot is a distributed web framework that allows programmers to create web applications in multiple programming languages
Go ☆ 631 35

invaders
Space Inva
Go

gonn
Building a
Go

260 contributions in the last year

Mar Apr May Jun Jul Aug Sep Oct Nov Dec Jan Feb Mar
Mon
Wed
Fri

Learn how we count contributions
Less More

Activity overview
Code review

2026
2025
2024
2023
2022
2021

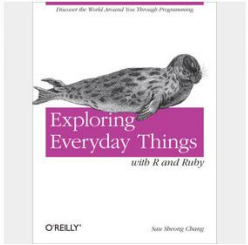
Active coder, 5 programming books, organizing community events

Sau Sheong Chang

Live online courses, books, and videos on O'Reilly

Books

Go Web Programming



Exploring Everyday Things with R and Ruby

Transformation

SP Group
6 yrs 2 mos

- **CEO, SP Digital**
  Full-time
  Feb 2019 - Jul 2022 · 3 yrs 6 mos

Before that, he ran SP Digital, the digital business unit of SP Group. He also headed IT, OT and digital transformation for the whole of SP Group for more than 6 years. Sau Sheong was instrumental in changing SP Group from a traditional utility company to a regional energy player.

- **SVP and Managing Director, Digital Technology**
  Jun 2016 - Feb 2019 · 2 yrs 9 mos
  Singapore

  I led the digital transformation of SP Group (previously known as Singapore Power), the leading energy utility in Asia Pacific and one of Singapore's largest corporations. I built a team to drive improved operational efficiency with new digital capabilities to explore new lines of business.

PayPal
2 yrs 10 mos
Singapore

Director, Global Consumer Engineering
Mar 2015 - Jun 2016 · 1 yr 4 mos

I lead a team of engineers to deliver Consumer products for PayPal, including the new generation of PayPal consumer app, partner onboarding integration, PayPal Digital Gifts and many others.

In his 28 years of industry experience, he has built and led product engineering teams at PayPal, Yahoo and HP building various software products for enterprise businesses and consumers globally.

Director, HP Labs Singapore
HP
Apr 2010 - Sep 2013 · 3 yrs 6 mos
Singapore

I built and lead a team of research scientists and engineers in HP Labs Singapore, the research arm of Hewlett-Packard. We research, innovate and build on cloud computing, big data and urbanization topics.

HP Labs Singapore was instrumental in clinching the first government-wide hybrid private-public cloud implementation in the world. Our technology powers the demand layer for the implementation and we are deeply involved in the execution and the implementation.

Startups

Chief Architect, VP Product Engineering (Co-Founder)
elipva Ltd
Dec 1999 - Nov 2003 · 4 yrs

I co-founded elipva in 1999 with funding from ST Telemedia. We started with the premise of a portal application framework (PAF) to help companies to build portals and e-businesses. By 2001 we had offices in Singapore, Malaysia and I

Experience in creating startups

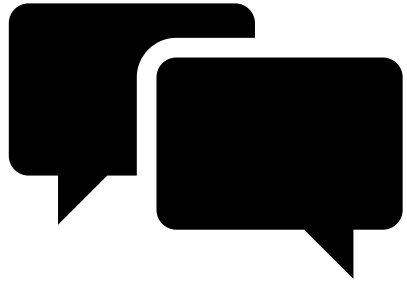
I managed the product engineering team as VP of Product and Chief Architect, designing and managing the product development of elipva's software products. Most of the technologies are Java and J2EE based, with some development on mobile platforms, specifically J2ME. Later development focused on intranets and improving internal business processes.

In 2004, elipva was acquired by MDream Inworld, a HK listed company.

Suggestions for Questions

- AI and the future of SE
- Job prospects: what skills are desired? What about the lower number of entry-level positions?
- Agile vs. plan-driven development (startups, engineering companies, tech companies)
- Government and tech: dependency to big tech in the US vs. sovereignity
- Sustainability and SE

Course Exercise



What questions should we ask him?

https://pollev.com/manuelrigger

Recap

Debugging

(Image by Gemini)

Debugging: Why?

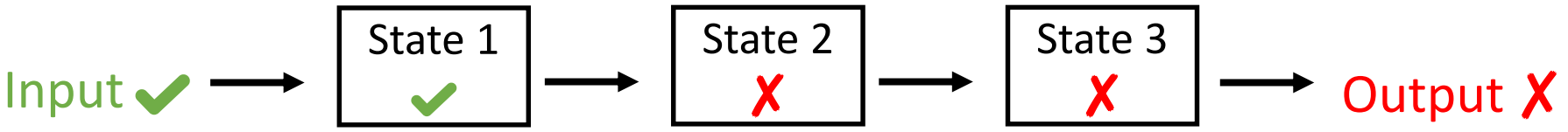
- Software developers spend **35–50% of their time validating and debugging software**
- Estimated **cost of debugging, testing, and verification is estimated to account for 50–75%** of the total budget of software development projects
- Conclusion: identifying systematic and effective approaches to debugging greatly enhances software engineering productivity

https://dl.acm.org/doi/pdf/10.1145/3055301.3068754

“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior
3. Determining the logical consequences of the hypothesis, **formulate a prediction that can support or refute the hypothesis**. Ideally, the prediction would distinguish the hypothesis from likely alternatives.
4. **Test the prediction** (and thus the hypothesis) in an experiment. If the prediction holds, confidence in the hypothesis increases; otherwise, it decreases.
5. Repeat Steps 2–4 until there are no discrepancies between hypothesis and predictions and/or observations.

Tracking Origins Debugging Strategy



Mistake → Defect → Fault → ... → Fault → Failure

- Debugging strategy: “tracking origins”
- We start with a single faulty state *f* – the failure.
- We determine *f*'s ***origins*** – the parts of earlier states that could have caused the faulty state *f*.
- For each of these origins *e*, we determine whether they are faulty or not, until we have found the defect.

Program Slicing

```
public static int middle(int x, int y, int z) {
    if (y < z) {
        if (x < y) {
            return y;
        } else if (x < z) {
            return y;
        }
    } else {
        if (x > y) {
            return y;
        } else if (x > z) {
            return x;
        }
    }
    return z;
}
```

```
int m = middle(2, 1, 3);
System.out.println(m);
```

A ***slice*** identifies the subset of the program that could have influenced the variable/returned value

Statistical Fault Localization: Tarantula

Line	abc ✔	abc ✔	"abc" ✗
1 (boolean tag = false;)	X	X	X
2	X	X	X
3	X	X	X
4	X	X	X
5	X	X	X
6	X	X	-
7	X	X	X
8 (tag = false;)	-	X	-
9	X	X	X
10 (quote = !quote;)	-	-	X
11	X	X	X
12	X	X	
15	X	X	

$$Suspiciousness(10) = \frac{\frac{failed(10)}{total\ failed}}{\frac{failed(10)}{total\ failed} + \frac{passed(10)}{total\ passed}} = \frac{\frac{1}{1}}{\frac{1}{1} + \frac{0}{2}} = 1$$

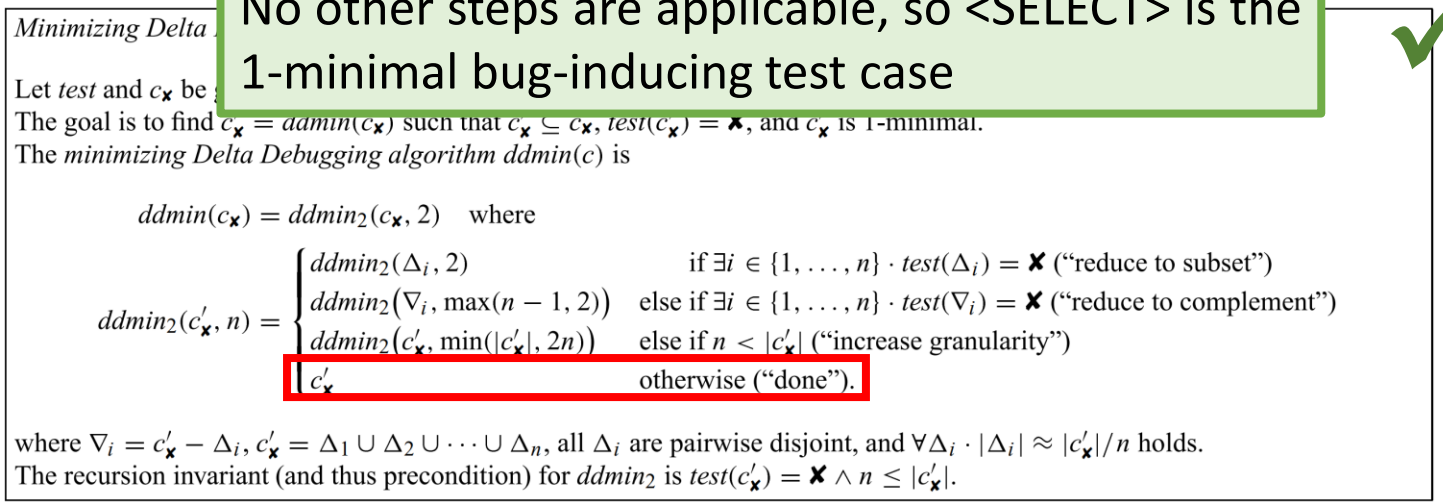
Highly suspicious line, as it was covered only in failing tests

$c_{\text{✘}}'$ = <SELECT>

Delta Debugging: Test-case Reduction

- Δ_1= <, Δ_2=S, Δ_3= E, Δ_4= L, Δ_5= E, Δ_6= C, Δ_7= T, Δ_8= >
- ∇_1= SELECT>, ∇_2= <ELECT>, ∇_3= <SLECT>, ∇_4= <SEECT>, ∇_5= <SELCT>, ∇_6= <SELET>, ∇_7= <SELEC> ∇_8= <SELECT

No other steps are applicable, so <SELECT> is the 1-minimal bug-inducing test case



Minimizing Delta

Let *test* and $c_{\text{✘}}$ be
The goal is to find $c_{\text{✘}}' = ddmin(c_{\text{✘}})$ such that $c_{\text{✘}}' \subseteq c_{\text{✘}}$, $test(c_{\text{✘}}') = \text{✘}$, and $c_{\text{✘}}'$ is 1-minimal.
The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_{\text{✘}}) = ddmin_2(c_{\text{✘}}, 2) \quad \text{where}$$

$$ddmin_2(c_{\text{✘}}', n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = \text{✘} \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = \text{✘} \text{ (“reduce to complement”)} \\ ddmin_2(c_{\text{✘}}', \min(|c_{\text{✘}}'|, 2n)) & \text{else if } n < |c_{\text{✘}}'| \text{ (“increase granularity”)} \\ c_{\text{✘}}' & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c_{\text{✘}}' - \Delta_i$, $c_{\text{✘}}' = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\text{✘}}'|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c_{\text{✘}}') = \text{✘} \wedge n \leq |c_{\text{✘}}'|$.

Program Slicing Example

```java
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

"I'm out at 2:00 AM. What chain of life choices led me here?"

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

"I'm out at 2:00 AM. What chain of life choices led me here?"

Forward or back-ward slicing?

Backward Slicing

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

"I'm out at 2:00 AM. What chain of life choices led me here?"

Backward Slicing

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

"I'm out at 2:00 AM. What chain of life choices led me here?"

Static or dynamic slicing?

Static Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

What is part of the static slice?

Static Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

How to compute a dynamic slice?

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

workDone = false, savings = 0, friendCount = 10

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

workDone = false, savings = 0, friendCount = 10

What is the dynamic slice?

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

workDone = false, savings = 0, friendCount = 10

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

Different from code coverage!

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

What is a question that forward slicing helps us answer?

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

If we finish the CS3213 assignment by Friday, how does that change my weekend?

Dynamic Slice

```
public boolean decideWeekendPlan(boolean workDone, double savings, int friendCount) {
    boolean canAfford = savings > 30.0;                    // Line 1
    boolean peerPressure = friendCount > 5;                // Line 2
    boolean willGoOut = false;                             // Line 3
    if (workDone) {                                        // Line 4
        if (canAfford || peerPressure) {                   // Line 5
            willGoOut = true;                              // Line 6
        }
    } else if (peerPressure) {                             // Line 7 (FOMO logic)
        willGoOut = true;                                  // Line 8
    }
    return willGoOut;                                      // Line 9
}
```

If we finish the CS3213 assignment by Friday, how does that change my weekend?

Delta Debugging Example

Example: C-Reduce

test.sql

interesting.sh

```
#!/bin/bash
sqlite3 < test.sql 2>&1 | grep -zq
"malformed“
```

creduce interesting.sh test.sql

Derive

git checkout version-3.28.0

```
CREATE VIRTUAL TABLE vt0 USING fts4(order=DESC);
REPLACE INTO vt0 VALUES ('0.5826273591290333'");
INSERT OR ABORT INTO vt0 VALUES (0.5674108139086818);
INSERT INTO vt0(vt0) VALUES('integrity-check')
```

https://github.com/csmith-project/creduce

Delta Debugging

Index	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Course	CS1010	CS2030	CS2103T	**CS3213**	CS2100	CS2106	**CS3213**	CS4226

Assumption: application crashes when processing NUS CS course list (we do not yet know why, but it is due to duplicate class codes)

Delta Debugging

Index	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Course	CS1010	CS2030	CS2103T	**CS3213**	CS2100	CS2106	**CS3213**	CS4226

Question: how many steps (calls to the interestingness test) do we need to derive a minimal bug-inducing example when using the delta-debugging algorithm?

c_x' = 1 2 3 4 5 6 7 8 n = 2

Delta Debugging: Test-case Reduction

calls = 4

- Δ_1= 1 2 3 4, Δ_2= 5 6 7 8
- $\nabla_1 = \Delta_2$, $\nabla_2 = \Delta_1$

Minimizing Delta Debugging Algorithm

Let *test* and c_x be given such that $test(\emptyset) = ✔ \wedge test(c_x) = ✘$ hold.
The goal is to find $c'_x = ddmin(c_x)$ such that
The *minimizing Delta Debugging algorithm*

Increase granularity, since no reductions apply

$$ddmin(c_x) = ddmin_2(c_x, 2) \quad \text{where}$$

$$ddmin_2(c'_x, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = ✘ \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = ✘ \text{ (“reduce to complement”)} \\ ddmin_2(c'_x, \min(|c'_x|, 2n)) & \text{else if } n < |c'_x| \text{ (“increase granularity”)} \\ c'_x & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_x - \Delta_i$, $c'_x = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_x|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_x) = ✘ \wedge n \leq |c'_x|$.

c_x' = 1 2 3 4 5 6 7 8 n = 4

Delta Debugging: Test-case Reduction

- Δ_1= 1 2, Δ_2=3 4, Δ_3= 5 6, Δ_4 7 8
- ∇_1= 3 4 5 6 7 8 ✗

calls = 5

Minimizing Delta Debugging Algorithm

Let *test* and c_x be given such tha
The goal is to find $c'_x = ddmin($
The *minimizing Delta Debuggin*

The first complement still fails the test, so continue with n = 3

$$ddmin(c_x) = ddmin_2(c_x, 2) \quad \text{where}$$

$$ddmin_2(c'_x, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = \text{✘ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = \text{✘ (“reduce to complement”)} \\ ddmin_2(c'_x, \min(|c'_x|, 2n)) & \text{else if } n < |c'_x| \text{ (“increase granularity”)} \\ c'_x & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_x - \Delta_i$, $c'_x = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_x|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_x) = \text{✘} \wedge n \leq |c'_x|$.

$c_✘'$ = 3 4 5 6 7 8 n = 3

Delta Debugging: Test-case Reduction

- Δ_1= 3 4, Δ_2=5 6, Δ_3= 7 8
- ∇_1= 5 6 7 8 , ∇_2= 3 4 7 8

✘

calls = 5

Minimizing Delta Debugging Algorithm

Let *test* and $c_✘$ be given such tha
The goal is to find $c'_✘ = ddmin($
The *minimizing Delta Debuggin*

The second complement still fails the test, so continue with n = 2

$$ddmin(c_✘) = ddmin_2(c_✘, 2) \quad \text{where}$$

$$ddmin_2(c'_✘, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = ✘ \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = ✘ \text{ (“reduce to complement”)} \\ ddmin_2(c'_✘, \min(|c'_✘|, 2n)) & \text{else if } n < |c'_✘| \text{ (“increase granularity”)} \\ c'_✘ & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_✘ - \Delta_i$, $c'_✘ = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_✘|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_✘) = ✘ \wedge n \leq |c'_✘|$.

c_x' = 3 4 7 8 n = 2

Delta Debugging: Test-case Reduction

- Δ_1= 3 4, Δ_2=7 8
- $\nabla_1 = \Delta_2$, $\nabla_2 = \Delta_1$

calls = 4

Minimizing Delta Debugging Algorith

Let *test* and c_x be given such that *test*(
The goal is to find $c_x' = ddmin(c_x)$ suc
The *minimizing Delta Debugging algo*

No reductions apply, so continue with n = 4

$$ddmin(c_x) = ddmin_2(c_x, 2) \quad \text{where}$$

$$ddmin_2(c_x', n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = \text{✘ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = \text{✘ (“reduce to complement”)} \\ ddmin_2(c_x', \min(|c_x'|, 2n)) & \text{else if } n < |c_x'| \text{ (“increase granularity”)} \\ c_x' & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c_x' - \Delta_i$, $c_x' = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_x'|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c_x') = \text{✘} \wedge n \leq |c_x'|$.

c_x' = 3 4 7 8 n = 4

Delta Debugging: Test-case Reduction

- Δ_1= 3, Δ_2=4, Δ_3= 7, Δ_4= 8
- ∇_1= 4 7 8

calls = 5

Minimizing Delta Debugging Algorithm

Let *test* and c_x be given such that $test(\emptyset) = ✔ \wedge test(c_x) = ✘$ hold.
The goal is to find $c'_x = ddmin(c_x)$ such that
The *minimizing Delta Debugging algorithm*

The first complement still triggers the bug, so continue with n =3

$$ddmin(c_x) = ddmin_2(c_x, 2) \quad \text{whe}$$

$$ddmin_2(c'_x, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = ✘ \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = ✘ \text{ (“reduce to complement”)} \\ ddmin_2(c'_x, \min(|c'_x|, 2n)) & \text{else if } n < |c'_x| \text{ (“increase granularity”)} \\ c'_x & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_x - \Delta_i$, $c'_x = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_x|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_x) = ✘ \wedge n \leq |c'_x|$.

c_x' = 4 7 8 n = 3

Delta Debugging: Test-case Reduction

- Δ_1= 4, Δ_2=7, Δ_3= 8
- ∇_1= 7 8, ∇_2= 4 8, ∇_3= 4 7

calls = 6

Minimizing Delta Debugging Algorithm

Let *test* and c_x be given such that $test(\emptyset) = ✔ \wedge test(c_x) = ✘$ hold.
The goal is to find c'_x =
The *minimizing Delta*

The third complement still reproduces the bug, so continue with n = 2

$$ddmin(c_x) = ddmin_2(c_x, 2)$$

$$ddmin_2(c'_x, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = ✘ \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = ✘ \text{ (“reduce to complement”)} \\ ddmin_2(c'_x, \min(|c'_x|, 2n)) & \text{else if } n < |c'_x| \text{ (“increase granularity”)} \\ c'_x & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_x - \Delta_i$, $c'_x = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_x|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_x) = ✘ \wedge n \leq |c'_x|$.

$c_{\text{✘}}' = \underline{4}\ \underline{7}\ n = 2$

Delta Debugging: Test-case Reduction

- $\Delta_1 = \underline{4}, \Delta_2 = \underline{7}$
- $\nabla_1 = \Delta_1, \nabla_2 = \Delta_2$

calls = 4

Minimizing Delta Debugging Algorithm

Let *test* and $c_{\text{✘}}$ be gi[illegible] such that *test*(∅) = ✔ ∧ *test*(c[illegible]) = ✘ hold
The goal is to find [illegible]
The *minimizing De*[illegible]

No more reductions possible, so 4 7 is our minimal bug-inducing test case!

ddmin([illegible])

$$ddmin_2(c'_{\text{✘}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = \text{✘} \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = \text{✘} \text{ (“reduce to complement”)} \\ ddmin_2(c'_{\text{✘}}, \min(|c'_{\text{✘}}|, 2n)) & \text{else if } n < |c'_{\text{✘}}| \text{ (“increase granularity”)} \\ c'_{\text{✘}} & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_{\text{✘}} - \Delta_i$, $c'_{\text{✘}} = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\text{✘}}|/n$ holds.
The recursion invariant (and thus precondition) for *ddmin*$_2$ is $test(c'_{\text{✘}}) = \text{✘} \wedge n \leq |c'_{\text{✘}}|$.

c_x' = 4 7 n = 2

Delta Debugging: Test-case Reduction

- Δ_1= 4, Δ_2= 7
- $\nabla_1 = \Delta_1$, $\nabla_2 = \Delta_2$

calls = 4 + 5 + 5 + 4 + 5 + 6 + 4

Overall, 33 interestingness tests

Minimizing Delta Debugging Alg

Let *test* and c_x be given such that $test(\emptyset) = ✔ \wedge test(c_x) = ✘$ hold.
The goal is to find $c'_x = ddmin(c_x)$ such that $c'_x \subseteq c_x$, $test(c'_x) = ✘$, and c'_x is 1-minimal.
The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_x) = ddmin_2(c_x, 2) \quad \text{where}$$

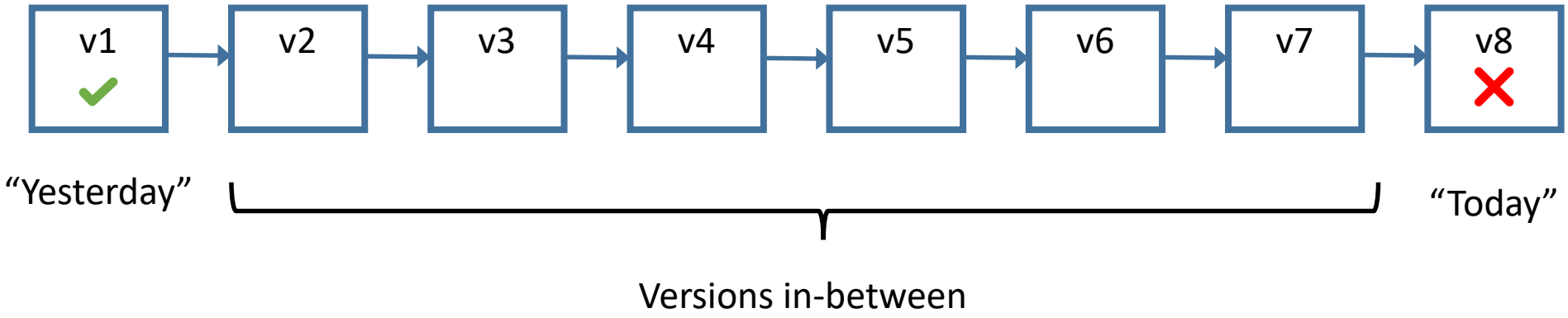
$$ddmin_2(c'_x, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = ✘ \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = ✘ \text{ (“reduce to complement”)} \\ ddmin_2(c'_x, \min(|c'_x|, 2n)) & \text{else if } n < |c'_x| \text{ (“increase granularity”)} \\ c'_x & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_x - \Delta_i$, $c'_x = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_x|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_x) = ✘ \wedge n \leq |c'_x|$.

Isolating Failure-Inducing Changes

Regression bugs. Git bisection. Delta debugging on patches.

Regression Bug



"Yesterday, my program worked.
Today, it does not. Why?"

Regression Bugs

- **Regression bugs**: something (e.g., a feature) that worked before, stopped working

How can we systematically (and automatically) debug regression bugs?

Using Git

Ticket Hash:	f3ff1472887cfba391c43de51ac2ea4a1761c9ee
Title:	Incorrect result for IN expression with right-hand IS TRUE sub-expression
Status:	Closed
Type:	Code_Defect
Severity:	Minor
Priority:	Low
Subsystem:	Unknown
Resolution:	Fixed
Last Modified:	2020-08-24 10:53:55
Version Found In:	

User Comments:

mrigger added on 2020-08-24 08:17:32:

The following expression unexpectedly evaluates to FALSE:

```
SELECT (1 IN (2 IS TRUE)); -- expected: {1}, actual: {0}
```

I found this based on commit [5f58dd3a19] and SQLancer's PQS implementation. It seems that this bug was introduced by commit [27936e6884].

---

dan added on 2020-08-24 10:5

Fixed by [493a2594].

We found the bug using SQLancer, but it could have been a user reporting the bug after a version upgrade

https://www.sqlite.org/src/tktview?name=f3ff147288

Using Git

git log | grep -B 10 5f58dd3a19

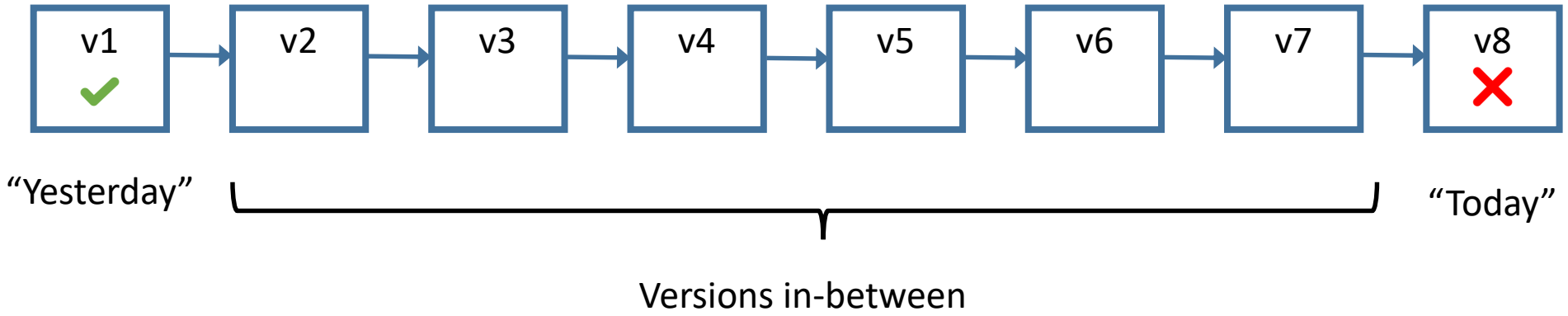
```
manuel@SOC-PC27WZ7S:~/sqlite$ git log | grep -B 10 5f58dd3a19
commit 6c3b4b07d13f1cb90471035823819 25a01f1a3a2
Author: dan <dan@noemail.net>
Date:   Thu Aug 20 16:25:26 2020 +0000

    Fix a crash that could occur in SQLITE_MAX_EXPR_DEPTH=0 builds when processing SQL containing syntax errors.

    FossilOrigin-Name: 5f58dd3a19605b6f49b4364fa29892502eff35f12a7693a8694100e1844711ea
```

SQLite uses another version control management system. We can find this systems' commit ID in its GitHub mirror using grep

Git: Checking out “Today’s” Version

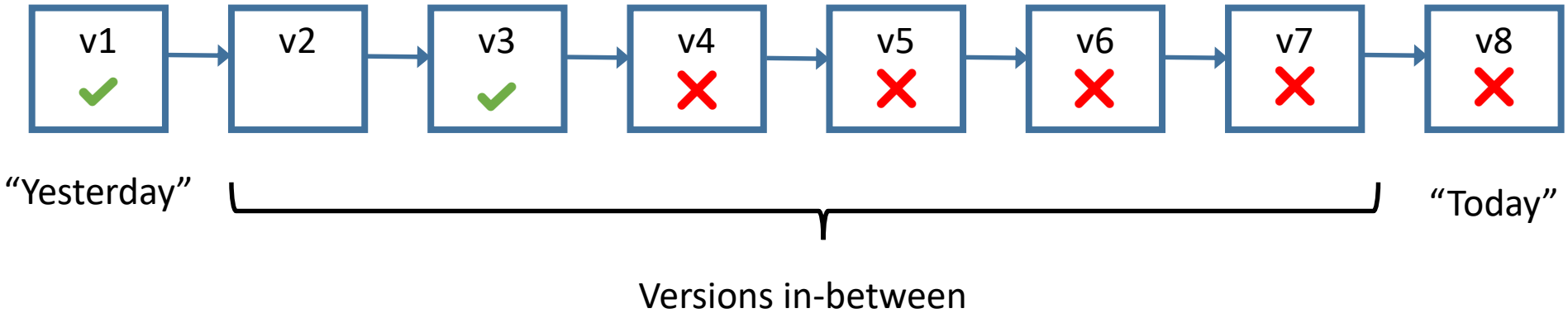
```
git checkout 6c3b4b07d13f1cb9047103582381925a01f1a3a2
./configure
make -j4
./sqlite3 test.db "SELECT (1 IN (2 IS TRUE));" -- 0
```

Automatic Regression Debugging



- **Prerequisite**: we have older, working versions of the software
- Typically, a version control systems like git

Naïve Approach

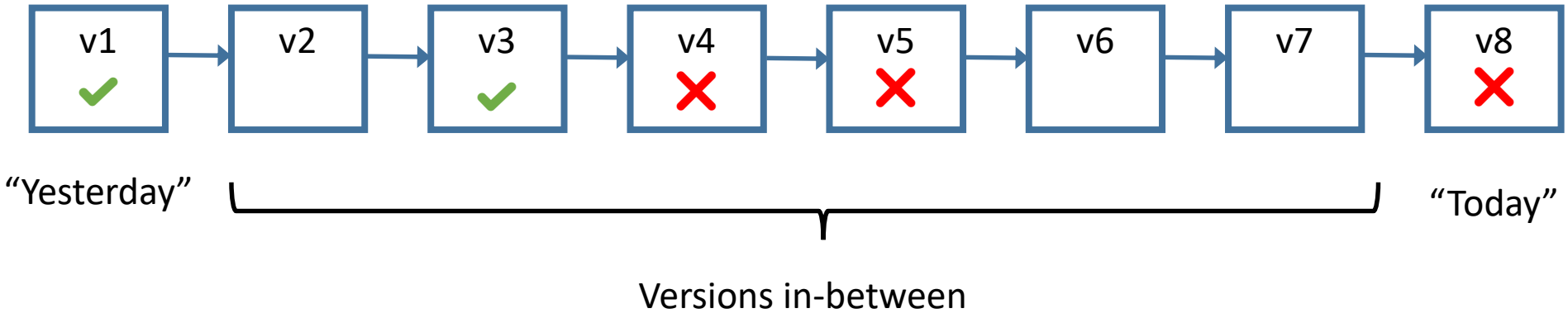


We could execute past versions version-by-version and check whether the failure can be triggered

Naïve Approach: Disadvantages

- **Would take a long time**: depending on the context, hundreds of versions might need to be inspected
- Between two adjacent versions, there might still be **many changes**
- This is a task that can actually be fully automated

Binary Search



Binary search is more efficient, as we can identify the commit in which the regression bug was introduced in log(n) tries

Git Bisect

SYNOPSIS

```
git bisect <subcommand> <options>
```

DESCRIPTION

The command takes various subcommands, and different options depending on the subcommand:

```
git bisect start [--term-(bad|new)=<term-new> --term-(good|old)=<term-old>]
          [--no-checkout] [--first-parent] [<bad> [<good>...]] [--] [<pathsp
git bisect (bad|new|<term-new>) [<rev>]
git bisect (good|old|<term-old>) [<rev>...]
git bisect terms [--term-(good|old) | --term-(bad|new)]
git bisect skip [(<rev>|<range>)...]
git bis
git bis
git bis
git bisect log
git bisect run <cmd> [<arg>...]
git bisect help
```

Git provides functionality to achieve this!

https://git-scm.com/docs/git-bisect

Finding a Possible “Yesterday’s” Version

2021-01-20	3.34.1
2020-12-01	3.34.0
2020-08-14	3.33.0
2020-06-18	3.32.3
2020-06-04	3.32.2
2020-05-25	3.32.1
2020-05-22	3.32.0
2020-01-27	3.31.1

```
git tag -list
git checkout version-3.31.1
make -j4
./sqlite3 test.db "SELECT (1 IN (2 IS TRUE));" -- 1
```

Git: Bisect

```
$ git bisect start
$ git bisect good version-3.31.1
$ git bisect bad 6c3b4b07d13f1cb9047103582381925a01f1a3a2
Bisecting: 264 revisions left to test after this (roughly 8 steps)
[e859e43bb87baa49cd3cdbbf81b4cbc9fe93d1de] Version number to 3.32.1.
```

Git: Bisect on the Intermediate Version

```
$ make -j4
$ ./sqlite3 test.db "SELECT (1 IN (2 IS TRUE));" – 0
$ git bisect bad
Bisecting: 132 revisions left to test after this (roughly 7 steps)
[054a0815554438116da171a18e4dfd8e50104543] Merge accidentally created fork.
```

Can we automate this process?

Automatic Bisection Using Git

- A test script to automate bisecting does the following:
  - It (re)builds the program under test for the given version
  - It tests whether the failure is present.
- Its exit code indicates the test outcome:
  - 0 means "good" (the failure did not occur)
  - 1 means "bad" (the failure did occur)
  - 125 means "undetermined" (we cannot decide if the failure is present or not)

Automatic Bisection Using Git

$ cat test.sh

```
#!/bin/sh
./configure
make -j4
output=$(./sqlite3 test.db "SELECT (1 IN (2 IS TRUE));")
if [ "$output" -eq 1 ]; then
    exit 0
else
    exit 1
fi
```

Automatic Bisection Using Git

```
$ git bisect start
$ git bisect good version-3.31.1
$ git bisect bad 6c3b4b07d13f1cb9047103582381925a01f1a3a2
$ git bisect run ./test.sh
```

```
95b395901a3b34c9b26b42da80ead52d02f0e54a is the first bad commit
commit 95b395901a3b34c9b26b42da80ead52d02f0e54a
Author: drh <drh@noemail.net>
Date:   Thu Mar 26 00:29:50 2020 +0000

    Reinstate the optimization that converts "x IN (y)" into "x==y".

    FossilOrigin-Name: 27936e6884e77093533719c7955a17f051cfb359872e51a6d1481152e6256443

 manifest      | 14 +++++++-------
 manifest.uuid |  2 +-
 src/expr.c    | 12 +++++---------
 src/parse.y   |  3 ++-
 4 files changed, 13 insertions(+), 18 deletions(-)
bisect run success
```

Automatic Bisection Using Git

```
--- a/src/expr.c
+++ b/src/expr.c
@@ -2941,6 +2941,8 @@ void sqlite3CodeRhsOfIN(
     affinity = sqlite3ExprAffinity(pLeft);
     if( affinity<=SQLITE_AFF_NONE ){
       affinity = SQLITE_AFF_BLOB;
+    }else if( affinity==SQLITE_AFF_REAL ){
+      affinity = SQLITE_AFF_NUMERIC;
     }
     if( pKeyInfo ){
       assert( sqlite3KeyInfoIsWriteable(pKeyInfo) );
@@ -3250,21 +3252,13 @@ static void sqlite3ExprCodeIN(
     int r2, regToFree;
     int regCkNull = 0;
     int ii;
-    int bLhsReal;  /* True if the LHS of the IN has REAL affinity */
     assert( !ExprHasProperty(pExpr, EP_xIsSelect) );
     if( destIfNull!=destIfFalse ){
       regCkNull = sqlite3GetTempReg(pParse);
       sqlite3VdbeAddOp3(v, OP_BitAnd, rLhs, rLhs, regCkNull);
     }
-    bLhsReal = sqlite3ExprAffinity(pExpr->pLeft)==SQLITE_AFF_REAL;
     for(ii=0; ii<pList->nExpr; ii++){
-      if( bLhsReal ){
-        r2 = regToFree = sqlite3GetTempReg(pParse);
-        sqlite3ExprCode(pParse, pList->a[ii].pExpr, r2);
-        sqlite3VdbeAddOp4(v, OP_Affinity, r2, 1, 0, "E", P4_STATIC);
-      }else{
-        r2 = sqlite3ExprCodeTemp(pParse, pList->a[ii].pExpr, &regToFree);
-      }
+      r2 = sqlite3ExprCodeTemp(pParse, pList->a[ii].pExpr, &regToFree);
       if( regCkNull && sqlite3ExprCanBeNull(pList->a[ii].pExpr) ){
         sqlite3VdbeAddOp3(v, OP_BitAnd, regCkNull, r2, regCkNull);
       }
```

$ git show 95b395901a3b34c9b26b42da80ead52d02f0e54a

```
--- a/src/parse.y
+++ b/src/parse.y
@@ -1194,10 +1194,11 @@ expr(A) ::= expr(A) between_op(N) expr(X) AND expr(Y). [BETWEEN] {
       */
       sqlite3ExprUnmapAndDelete(pParse, A);
       A = sqlite3Expr(pParse->db, TK_INTEGER, N ? "1" : "0");
-    }else if( 0 && Y->nExpr==1 && sqlite3ExprIsConstant(Y->a[0].pExpr) ){
+    }else if( Y->nExpr==1 && sqlite3ExprIsConstant(Y->a[0].pExpr) ){
       Expr *pRHS = Y->a[0].pExpr;
       Y->a[0].pExpr = 0;
       sqlite3ExprListDelete(pParse->db, Y);
+      pRHS = sqlite3PExpr(pParse, TK_UPLUS, pRHS, 0);
       A = sqlite3PExpr(pParse, TK_EQ, A, pRHS);
       if( N ) A = sqlite3PExpr(pParse, TK_NOT, A, 0);
     }else{
```

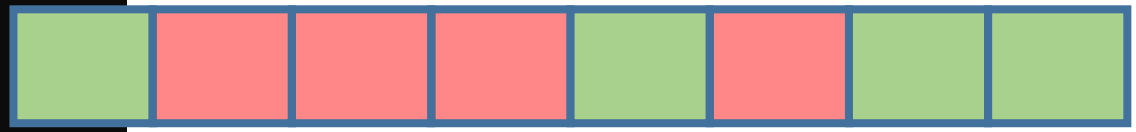
Can we further narrow down the defect?

Delta Debugging on Changes

- We can apply delta debugging by considering individual *patches* within a commit
- Different granularities possible
  - *Hunk*: consecutive differing lines (as opposed to lines common to both files)

Delta Debugging on Changes

```
--- a/src/expr.c
+++ b/src/expr.c
@@ -2941,6 +2941,8 @@ void sqlite3CodeRhsOfIN(
     affinity = sqlite3ExprAffinity(pLeft);
     if( affinity<=SQLITE_AFF_NONE ){
       affinity = SQLITE_AFF_BLOB;
+    }else if( affinity==SQLITE_AFF_REAL ){
+      affinity = SQLITE_AFF_NUMERIC;
     }
     if( pKeyInfo ){
       assert( sqlite3KeyInfoIsWriteable(pKeyInfo) );
@@ -3250,21 +3252,13 @@ static void sqlite3ExprCodeIN(
     int r2, regToFree;
     int regCkNull = 0;
     int ii;
-    int bLhsReal;  /* True if the LHS of the IN has REAL affinity */
     assert( !ExprHasProperty(pExpr, EP_xIsSelect) );
     if( destIfNull!=destIfFalse ){
       regCkNull = sqlite3GetTempReg(pParse);
       sqlite3VdbeAddOp3(v, OP_BitAnd, rLhs, rLhs, regCkNull);
     }
-    bLhsReal = sqlite3ExprAffinity(pExpr->pLeft)==SQLITE_AFF_REAL;
     for(ii=0; ii<pList->nExpr; ii++){
-      if( bLhsReal ){
-        r2 = regToFree = sqlite3GetTempReg(pParse);
-        sqlite3ExprCode(pParse, pList->a[ii].pExpr, r2);
-        sqlite3VdbeAddOp4(v, OP_Affinity, r2, 1, 0, "E", P4_STATIC);
-      }else{
-        r2 = sqlite3ExprCodeTemp(pParse, pList->a[ii].pExpr, &regToFree);
-      }
+      r2 = sqlite3ExprCodeTemp(pParse, pList->a[ii].pExpr, &regToFree);
       if( regCkNull && sqlite3ExprCanBeNull(pList->a[ii].pExpr) ){
         sqlite3VdbeAddOp3(v, OP_BitAnd, regCkNull, r2, regCkNull);
       }
```



```
--- a/src/parse.y
+++ b/src/parse.y
@@ -1194,10 +1194,11 @@ expr(A) ::= expr(A) between_op(N) expr(X) AND expr(Y). [BETWEEN] {
       */
       sqlite3ExprUnmapAndDelete(pParse, A);
       A = sqlite3Expr(pParse->db, TK_INTEGER, N ? "1" : "0");
-    }else if( 0 && Y->nExpr==1 && sqlite3ExprIsConstant(Y->a[0].pExpr) ){
+    }else if( Y->nExpr==1 && sqlite3ExprIsConstant(Y->a[0].pExpr) ){
       Expr *pRHS = Y->a[0].pExpr;
       Y->a[0].pExpr = 0;
       sqlite3ExprListDelete(pParse->db, Y);
+      pRHS = sqlite3PExpr(pParse, TK_UPLUS, pRHS, 0);
       A = sqlite3PExpr(pParse, TK_EQ, A, pRHS);
       if( N ) A = sqlite3PExpr(pParse, TK_NOT, A, 0);
     }else{
```

Based on this, we can now apply the delta debugging algorithm as before!